

Übungsblatt 4

Ü 4.1 Classical IPC Problems

Recherchieren Sie das „**Readers/Writers**“-Problem und beschreiben Sie dieses mit eigenen Worten. Wie unterscheidet sich dieses Problem von den anderen bekannten, klassischen IPC-Problemen (z.B. **Dining Philosopher**, **Sleeping Barber**, **Cigarette Smokers**)?

Ü 4.2 Erzeugung von Prozessen mit `fork()`

In der **Vorlesung** haben Sie sich bereits intensiv mit dem `fork()`-Systemcall und seinen Rückgabewerten auseinandergesetzt. Was ist der **Rückgabety**p von `fork()` und wie wird der zurückgegebene Wert interpretiert?

Stellen Sie die Eltern-Kind-Beziehungen des folgenden C-Sourcecodes in Form eines Baumes dar, in dem ersichtlich ist, wie viele Prozesse erzeugt werden (und durch welchen Prozess ein anderer Prozess erzeugt wurde).

```
for (int i = 0; i < 2; i++)
{
    if (fork() == 0)
    {
        printf("Outer Hello World\n");
        for (int j = 0; j < 2; j++)
        {
            pid = fork();
            printf("-- Inner Hello World\n");
            if (pid == 0)
            {
                exit(0);
            }
        }
    }
}
```

Überlegen Sie sich zuerst, wie der Prozessbaum aussieht (z.B. per Papier & Bleistift). Implementieren Sie dann ein vollständiges C-Programm, das das obenstehende Fragment enthält, und schauen Sie sich z.B. mit `ps` oder `top` an, welche Prozesse entstehen und wie sie tatsächlich zusammenhängen. Fügen Sie im Programm entsprechende „sleeps“ (`#include<unistd.h> sleep(10);`) ein, um das Verhalten besser beobachten zu können.

Ü 4.3 Prozesse

Schreiben Sie das Programm `myforking.c`. Erstellen Sie fünf Kind-Prozesse von einem Eltern-Prozess. Geben Sie nach dem `fork()` eines Kindprozesses seine Prozess-ID und die Prozess-ID des Elternprozesses aus.

Zusätzlich soll jeder Kind-Prozess von 1 bis zur Prozess-ID zählen und dies mittels `printf(...)` ausgeben. Die Ausgabe auf der Konsole sollte ungefähr so aussehen:

Beispiel mit einem Kind Prozess: <i>[kind] pid 6133 von [eltern] pid 5921</i> <i>[kind] pid 6133 Zähler: 6126</i> <i>[kind] pid 6133 Zähler: 6127</i> <i>[kind] pid 6133 Zähler: 6128</i> <i>[kind] pid 6133 Zähler: 6129</i> <i>[kind] pid 6133 Zähler: 6130</i> <i>[kind] pid 6133 Zähler: 6131</i> <i>[kind] pid 6133 Zähler: 6132</i> <i>[kind] pid 6133 Zähler: 6133</i> <i>[kind] pid 6133 zählt bis 6133</i>	Beispiel mit 5 Kind Prozess: (Zählen nicht in Ausgabe) <i>[kind] pid 6568 von [eltern] pid 6564</i> <i>[kind] pid 6568 zählt bis 6568</i> <i>[kind] pid 6569 von [eltern] pid 6564</i> <i>[kind] pid 6569 zählt bis 6569</i> <i>[kind] pid 6571 von [eltern] pid 6564</i> <i>[kind] pid 6571 zählt bis 6571</i> <i>[kind] pid 6570 von [eltern] pid 6564</i> <i>[kind] pid 6570 zählt bis 6570</i> <i>[kind] pid 6572 von [eltern] pid 6564</i> <i>[kind] pid 6572 zählt bis 6572</i>
--	--

Erklären Sie, warum manche Prozesse bereits beendet wurden, bevor andere überhaupt zu zählen begonnen haben.

Ü 4.4 Race Conditions

Erweitern Sie **Beispiel 4.3** um folgende Funktionalität: Erstellen Sie **20** Kind-Prozesse, die so oft eine Zahl inkrementieren, wie ihre eigene Prozess-ID. Das Zählen soll aber nicht mittels lokaler Variablen erfolgen, sondern über eine globale Variable, die außerhalb der `main()` Funktion deklariert wird (d.h., ein Prozess mit der ID 3 würde die globale Variable insgesamt 3-mal um 1 erhöhen). Die Ausgabe könnte wie folgt aussehen:

```
[kind] pid 19196 von [eltern] pid 19184  
[kind] pid 19195 zählt bis 228367  
[kind] pid 19191 zählt bis 209213  
[kind] pid 19194 zählt bis 228343  
[kind] pid 19200 von [eltern] pid 19195  
[kind] pid 19197 von [eltern] pid 19153  
[kind] pid 19198 zählt bis 266705  
[kind] pid 19169 zählt bis 209130  
[kind] pid 19190 zählt bis 209212  
[kind] pid 19185 zählt bis 209173  
[kind] pid 19172 zählt bis 209157  
[kind] pid 19202 von [eltern] pid 19176
```

Beantworten Sie die Fragen basierend auf Ihrem eigenen Code oder der obigen Ausgabe:

Fragen:

- 1) Terminiert das Programm regulär?
- 2) Wenn die PIDs aller Prozesse bei ~19.000 liegen und wir 20 Prozesse haben, sollte der globale Count bei 380.000 liegen. Warum erreichen wir diesen wahrscheinlich nicht einmal annähernd?
- 3) Welche Möglichkeiten gibt es, den Zugriff auf diese globale Variable zu sichern?

Ü 4.5 Core Dump GDB

Recherchieren Sie den Begriff core dump und beschreiben Sie kurz, aber mit eigenen Worten, was ein core dump ist. Müssen besondere Vorkehrungen getroffen werden, um einen core dump zu erhalten? Recherchieren Sie weitere Informationen über den GNU Debugger (GDB) und beschreiben Sie kurz, wozu dieser dient. Mit diesem Übungsblatt erhalten Sie das C-Programm `compute.c`, das einen Programmierfehler enthält. Zeigen Sie anhand dieser Implementierung, wie man bei Absturz des Programmes einen core dump erzeugen kann und diesen mit dem GDB analysiert:

- a) Welche Dateien benötigen Sie, um einen Core Dump analysieren zu können?
- b) Zeigen Sie im GDB die Codezeile (+ Umgebung), in der das Programm abgestürzt ist.
- c) Aus welchem Grund ist das Programm abgestürzt? Geben Sie im GDB die Werte der verursachenden Variablen aus.
- d) Geben Sie auch den Wert der Konstanten „NUM“ aus. Begründen Sie das Ergebnis.
- e) Geben Sie einen full Backtrace aus.